# Identity Demystified

Liran Ma and Zhipeng Cai
Department of Computer Science
Texas Christian University

Copyright © 2021 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

## Contents

**1 Lab Prep** . . . . . **1**

**2 Certification Verification** . . . . . **3**

**3 Public Key Application: Certificates** . . . . . **7**

**4 Configure Certificate on a Web Server (HTTPS)** . . . . . **10**

**5 Install CA Certificate on Clients** . . . . . **11**

**6 General Questions** . . . . . **14**

## 1 Lab Prep

In this lab, you can use Docker containers on your local computer or a virtual machine on the cloud (e.g., Amazon Web Services, AWS, https://www.awseducate.com/signin/SiteLogin).

### 1.1 Docker Container on Local Computer

Install Docker Desktop on your computer if you have not done so. It can be downloaded via https://www.docker.com/products/docker-desktop. Docker Desktop is free, but you may need to register before downloading it.

Make a folder (say COSC40053-Labs) to store all the lab related files. Enter the folder and download the GitHub repository that contains the Docker configuration files for this lab.

```
git clone https://github.com/liranmatcu/Docker_Security_Labs.git
```

Once you have cloned the repository, you can use "git pull" to get updated configuration files. It is strongly recommended that you do "git pull" before each lab task.

First, you need to build custom-build Docker image (say *sec-linux*). Next, start a container instance (say *ciphers*) using the sec-linux image. Example commands are given as follows.

```
cd Docker_Security_Labs/ciphers

docker build -t sec-linux .  # Note that there is a dot in the end

pwd    # Write down the full path of your current folder

docker run -it --name ciphers -v "$PWD"/local_data:/docker_data sec-linux bash
                                  or
docker run -it --name ciphers -v "$PWD"/local_data:/docker_data --rm sec-linux bash
```

Here, "$PWD" refers to the current folder. The *pwd* command will show the full path of your current folder. You will need to use it in the later sections. **Note that If you are using a virtual machine (e.g., Amazon EC2 on AWS) as a non-root user (ec2-user/ubuntu), you need to add sudo before almost all commands.**

When you work on configuring SSL/TLS on a web server in Section 4, you will start another container instance serving as the web server. Detailed commands will be provided over there.

## 1.2 Docker Container on AWS

You can log onto your virtual machine using ssh as follows.

```
ssh  -i  ``path-to-your-pem-file''  user-name@IP-address
```

The default user name is **ubuntu** for Ubuntu and **ec2-user** for AWS EC2. These users can perform administrative commands via sudo. You can become the root user by "**sudo su -**". **If you prefer to remain as ec2-user/ubuntu, you need to add sudo before almost all commands.**

To use Docker containers on an AWS Linux virtual machine (say Ubuntu), you will need to install it first. An example tutorial can be found here: https://geekylane.com/install-docker-on-aws-ec2-ubuntu-18-04-script-method/. The key commands are listed below.

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker ubuntu
```

After installing Docker, you can follow the steps in Section 1.1 to build your containers for this lab.

# 2 Certification Verification

Certificate verification are typically done automatically without user intervention (as you have experienced in the previous lab tasks). In this section, we will manually verify a certificate to gain an in-depth understanding the concept of chain of trust.

## 2.1 Download Certificates

To perform manual certificate verification, we need to download a web site's certificate and its issuer's certificate (i.e., the CA). You can download these two certificates using a command line utility (openssl) or browser (e.g., Firefox). For the sake of demonstration, we choose to use the Wikipedia site as an example. **Note that the required certificates have been loaded into our Docker image. Thus, the following instructions are provided for your reference.**

**Command line utility option**

Obtain Wikipedia's certificate and its issuer's (signer) certificate with the openssl utility as follows.

```
openssl s_client -showcerts -connect www.wikipedia.org:443 </dev/null >wiki-2.crt
```

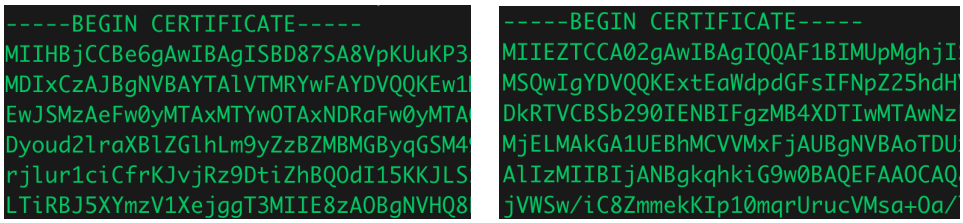
This command will download both certificates together and save them into the file name wiki-2.crt. You can view the two certificates via the cat command.

```
cat wiki-2.crt
```

The first certificate is the Wikipedia's (Fig. 1a) and the second one is its issuer's (Fig. 1b). Next, extract the two certificates from wiki-2.crt into two different files by "copy and paste" whole data area of inside and including the key words BEGIN CERTIFICATE and END CERTIFICATE. You can name the first certificate file as wikipedia-org.pem and the second certificate file as CA.pem (since the following instructions are based on this naming scheme).

**Browser options**

Another way to download a certificate is using Firefox. Go to https://www.wikipedia.org, and click on the padlock next to the link. It should be saying "connection is secure" in green. Click on the arrow next to it and "more information". You will find see "view certificate" button as shown




(a) Wikipedia certificate (partial). (b) CA (Issuer) certificate (partial).

Figure 1: Two certificates.

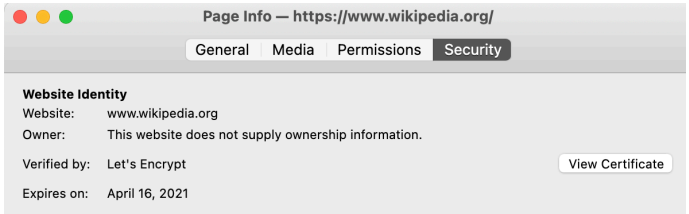



Figure 2: View the certificate of Wikipedia.

in Fig. 2.

Move to the **Miscellaneous** section, and click on **PEM (cert)** to download the certificate (as shown in Fig. 3).

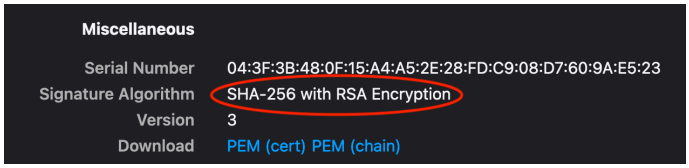



Figure 3: Download the certificate of Wikipedia.

Next, we will download CA's certificate. Inside Wikipedia's certificate (e.g., a PEM file; here it is wikipedia-org.pem), there is a link to download the CA's certificate. On your Docker instance, run the following commands to extract relevant information from the Wikipedia's certificate.

```
cd /root/wiki
openssl x509 -in wikipedia-org.pem -text -noout
```

In the **Authority Information Access** section, and it contains to URL to the CA Issuer's URL, which is http://r3.i.lencr.org/ for our example. You can use *wget* (or a browser) to download the **.crt** file. An example wget command is given below. The ca.crt also has been included in our Docker image.

```
wget -O ca.crt http://r3.i.lencr.org/
```

Since the downloaded file (ca.crt) is in **.crt** format, we need to change it into **.pem**.

```
openssl x509 -in ca.crt -inform der -outform pem -out CA.pem
```

To understand the different types of certificate extension, check out this reference: https://www.ssl.com/guide/pem-der-crt-and-cer-x-509-encodings-and-conversions/.

## 2.2 Extract TBS Field

TBSCertificate (TBS implies To-Be-Signed) is a basic certificate field that is a sequence which contains information associated with the subject of the certificate and the CA that issued it. TBSCertificate contains the data that are used for the certificate signature (Digital Signature) calculation, which is encoded using the ASN.1 Distinguished Encoding Rules (DER) X.690.

First of all, we need to locate the TBSCertificate field inside the certificate (i.e., starting location and ending location).

```
openssl asn1parse -i -in wikipedia-org.pem
# Make sure that you are in the /root/wiki folder,
# where wikipedia-org.pem is located.
openssl x509 -in wikipedia-org.pem -noout -text
```

The first a few lines of the command output are shown in Fig. 4. Based on certificate's ANS.1 definition, the second line represents that TBSCertificate's offset is 4, its total size is 1522 (head length 4, body length 1518).

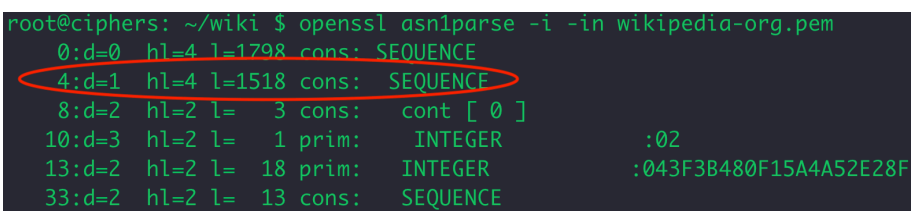



Figure 4: First few lines of Wikipedia certificate.

So it can be extracted via the following command:

```
openssl asn1parse -in wikipedia-org.pem -strparse 4 -out wikipedia.tbs
# Number 4 in this command is the offset found above.

# Or if you want to use the dd command
openssl x509 -outform der -in wikipedia-org.pem -out wikipedia-org.crt
dd if=wikipedia-org.crt of=wikipedia.tbs skip=4 bs=1 count=1522
```

Now, the TBSCertificate field has been extracted and placed into the file named wikipedia.tbs. We can calculate its signature as follows.

```
sha256sum wikipedia.tbs
```

The sha256 sum is "24498af72add3353fa2c3269e7c3d281d000f437341b1742d00cf6111ca52132" in our example (see Fig. 5). The reason that we use sha256 can be seen in Fig. 3. Note that other sha algorithms could be used by different issuers.

```
root@ciphers: ~/wiki $ sha256sum wikipedia.tbs
24498af72add3353fa2c3269e7c3d281d000f437341b1742d00cf6111ca52132  wikipedia.tbs
```

Figure 5: sha256 sum of Wikipedia certificate TBS field.

### 2.3 Extract Signature

Here, we will calculate the signature of the certificate manually. If correct, it should match the sha256 sum obtained in the previous section.

First, extract the signature (s) of the Wikipedia certificate as follows.

```
openssl x509 -in wikipedia-org.pem -noout -text
```

The extracted signature is shown in Fig. 6a.

### 2.4 Reverse Signature

Next, extract the modulus and exponent values provided by the issuer to "reverse" (i.e., decryption with the matching pubic key) the signature to the original sha256sum value as follows. Modulus (n) is the product of two prime numbers used to generate the key pair. Public exponent (d) is the exponent used on signed/encoded data to decode the original value. ($sha256sum = (s)^d \mod n$)

```
openssl x509 -in CA.pem -noout -text
```

The extracted values are shown in Fig. 6b.

Now, we can use a Python program to calculate the sha256sum with the obtained signature, modulus and exponent. The calculation procedure is shown in Fig. 7. The leading "1fff..." of the result is the standard RSA algorithm padding, following "3031300d0609608648016503040201050004 20" is the ASN.1 BER SHA256 algorithm designator prefix, so the actual decoded SHA1 hash value is the remaining "24498af72add3353fa2c3269e7c3d281d000f437341b1742d00cf6111ca52132". The Python program is included in the Docker image as well.

```
Signature Algorithm: sha256WithRSAEncryption
     92:9c:83:f4:a5:11:6e:c6:13:d5:d9:42:33:cd:5d:77:77:6b:
     9b:04:b9:eb:9f:d3:eb:38:4f:f7:6f:86:07:fd:42:94:d8:a6:
     fe:ff:73:00:86:1f:98:38:66:fe:a3:b3:97:7d:83:c3:67:5d:
     62:14:49:5c:db:09:be:72:e5:88:38:3a:c6:65:a2:62:25:de:
     9b:84:a9:11:19:b6:96:89:e5:dd:ed:1e:a0:78:f1:36:de:81:
     6f:50:f8:ac:03:4d:b5:2c:05:56:15:af:e4:57:60:c1:53:1e:
     23:84:41:70:57:01:f5:69:7b:49:fe:c2:79:f9:ed:63:c9:77:
     d9:f2:9a:86:ae:cc:95:9b:c4:d8:fc:a6:94:52:83:39:3c:6a:
     e3:d3:53:ed:2b:b0:bb:f4:8c:85:87:13:ca:59:bd:2d:a7:9e:
     44:9d:0e:7a:9e:03:0d:bc:8a:a7:c8:17:86:27:83:7e:27:33:
     04:18:37:bc:ac:4b:ed:7e:c5:f5:39:7c:d7:0b:aa:6c:41:2e:
     82:96:f9:e5:69:c2:24:47:20:9c:63:98:21:67:9b:38:da:5e:
     7d:2c:42:9a:da:c3:52:b4:9b:93:b5:33:92:c9:94:98:62:1c:
     7e:f9:0f:90:87:e4:f5:50:0c:e2:6e:c8:34:ad:14:8b:fa:64:
     38:b2:de:96
```

(a) Signature of Wikipedia certificate.

```
RSA Public-Key: (2048 bit)
Modulus:
    00:bb:02:15:28:cc:f6:a0:94:d3:0f:12:ec:8d:55:
    92:c3:f8:82:f1:99:a6:7a:42:88:a7:5d:26:aa:b5:
    2b:b9:c5:4c:b1:af:8e:6b:f9:75:c8:a3:d7:0f:47:
    94:14:55:35:57:8c:9e:a8:a2:39:19:f5:82:3c:42:
    a9:4e:6e:f5:3b:c3:2e:db:8d:c0:b0:5c:f3:59:38:
    e7:ed:cf:69:f0:5a:0b:1b:be:c0:94:24:25:87:fa:
    37:71:b3:13:e7:1c:ac:e1:9b:ef:db:e4:3b:45:52:
    45:96:a9:c1:53:ce:34:c8:52:ee:b5:ae:ed:8f:de:
    60:70:e2:a5:54:ab:b6:6d:0e:97:a5:40:34:6b:2b:
    d3:bc:66:eb:66:34:7c:fa:6b:8b:8f:57:29:99:f8:
    30:17:5d:ba:72:6f:fb:81:c5:ad:d2:86:58:3d:17:
    c7:e7:09:bb:f1:2b:f7:86:dc:c1:da:71:5d:d4:46:
    e3:cc:ad:25:c1:88:bc:60:67:75:66:b3:f1:18:f7:
    a2:5c:e6:53:ff:3a:88:b6:47:a5:ff:13:18:ea:98:
    09:77:3f:9d:53:f9:cf:01:e5:f5:a6:70:17:14:af:
    63:a4:ff:99:b3:93:9d:dc:53:a7:06:fe:48:85:1d:
    a1:69:ae:25:75:bb:13:cc:52:03:f5:ed:51:a1:8b:
    db:15
Exponent: 65537 (0x10001)
```

(b) CA modulus and exponent.

Figure 6: Signature, modulus and exponent extraction.

```
>>> signature=0x929c83f4a5116ec613d5d94233cd5d77776b9b04b9eb9fd3eb384ff76f8607fd4294d8a6feff7300861f98
be72e588383ac665a26225de9b84a91119b69689e5dded1ea078f136de816f50f8ac034db52c055615afe45760c1531e238441
9a86aecc959bc4d8fca6945283393c6ae3d353ed2bb0bbf48c858713ca59bd2da79e449d0e7a9e030dbc8aa7c8178627837e27
412e8296f9e569c22447209c639821679b38da5e7d2c429adac352b49b93b53392c99498621c7ef90f9087e4f5500ce26ec834
>>> modulus=0x00bb021528ccf6a094d30f12ec8d5592c3f882f199a67a4288a75d26aab52bb9c54cb1af8e6bf975c8a3d70f
4e6ef53bc32edb8dc0b05cf35938e7edcf69f05a0b1bbec094242587fa3771b313e71cace19befdbe43b45524596a9c153ce34
97a540346b2bd3bc66eb66347cfa6b8b8f572999f830175dba726ffb81c5add286583d17c7e709bbf12bf786dcc1da715dd446
e653ff3a88b647a5ff1318ea9809773f9d53f9cf01e5f5a6701714af63a4ff99b3939ddc53a706fe48851da169ae2575bb13cc
>>> print "%x" % pow( signature, 65537, modulus )
1ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
031300d06096086480165030402010500042024498af72add3353fa2c3269e7c3d281d000f437341b1742d00cf6111ca52132
```

Figure 7: Python code to calculate sha256sum $= (s)^d \mod n$.

### 2.4.1 Provide screenshots of your verification results of another web site of your choice (say www.google.com).

# 3 Public Key Application: Certificates

In this section, we will create two certificates. One is the certificate for a certificate authority (CA), and the other is for a website. The CA's certificate will be used to endorse (sign) the website certificate. Reference: http://www.reppep.com/~pepper/writing/tidbits/ssl-article/ssl2.text.

On your virtual environment (i.e., Docker instance), as the root user, go to the **/etc/ssl/misc/** directory to locate a file named **CA.pl**. You will need to use this file to create the CA's certificate and other certificates. Reference on how to use CA.pl is here: http://www.openssl.org/docs/apps/CA.pl.html.

## 3.1 Create a Root Certificate for CA

On your Docker instance, issues the following commands.

```
cd /etc/ssl/misc/
./CA.pl -newca
```

```
A certificate filename (or enter to create): Hit enter

Making CA certificate ...

writing new private key to `../../CA'
Enter PEM pass phrase: Type your secret passphrase
Write it down for future use, mine is cosc40053

You are about to be asked to enter information that will be
incorporated into your certificate request.

-----
Country Name (2 letter code): US
State or Province Name: TX
Locality Name (eg, city): Fort Worth
Organization Name (eg, company): TCU
Organizational Unit Name (eg, section): CS
Common Name (eg, YOUR name): CS_TCU
Note: I put CS_TCU here as an example common name. A general
      practice is to put an organization's name or DNS name.

Email Address: super.frog@tcu.edu
No need to add a challenge password
You will be asked to input your PEM key again in the end
(before the accomplish of the certificate creation)
```

This creates a new root certificate for CA, named as **cacert.pem**, placed in the **/etc/ssl/misc/demoCA/** folder. This certificate will be used to sign public keys to make them become certificates.

### 3.2 Generate a New Certificate Request

Next, we will create a certificate request for a web server. It is important that the certificate matches the web server's information such as the IP address (or DNS name). When using a Docker container on your local machine, the common name should be its loopback address, which is **127.0.0.1**. When using a Docker container on an AWS virtual machine (VM), the common name should be **the VM's public IP address**.

A new certificate request can be generated using the following command.

```
./CA.pl -newreq
```

```
writing new private key to `newreq.pem'
Enter PEM pass phrase: Type your secret passphrase here
It should be different from CA passphrase, mine is 40053req

-----

Organizational Unit Name (eg, section): WebDemo
Common Name (eg, YOUR name): 127.0.0.1
For local Docker users, use the IP address of 127.0.0.1
For AWS Docker users, you should use the public IP address of the VM.
If the web server has a real DNS name (FQDN), you should use the DNS name.

-----
A challenge password []: just hit enter to skip
An optional company name []: just hit enter to skip
Request (and private key) is in newreq.pem
```

The newly generated request is in **newreq.pem**, and its private key is in **newkey.pem**. Both files are located in the */etc/ssl/misc/* folder.

### 3.3 Sign the Certificate Request

We sign (endorse) the newly created request using the CA's certificate.

```
./CA.pl -sign
```

You will be prompted for the CA's certificate password, which is *cosc40053* in my example. If you received no errors, the certificate has been signed by your CA's certificate successfully. The signed certificate is in **newcert.pem**, which is also located in the **/etc/ssl/misc/** folder.

### 3.4 Remove Protection on Private Key

The access to the private key file (i.e., newkey.pem) is password protected (this is a common security measure to protect the private key). You will need to remove the protection on the private key file so that it can be used by the web server with ease. Here are the steps:

```
cd /etc/ssl/misc/
mv newkey.pem newkey.pem.org
openssl rsa -in newkey.pem.org -out newkey.pem
```

### 3.5 Transport Certificate Files to Local/Host Computer

The website certificate **newcert.pem** and its private key **newkey.pem**, which both are in the **/etc/ssl/misc/** directory on your Docker instance, need to be transported to the intended web

server.

We may want to rename the certificate files to intuitive names so as to avoid confusion. Specifically, rename **newcert.pem** and **newkey.pem** to **server.crt** and **server.key**, respectively. Next, copy the files to the mounted (shared) directory on the Docker container so they can be accessed from the host machine (i.e., your local computer or AWS VM).

```
mv newcert.pem server.crt
mv newkey.pem server.key
cp  server.*  /docker_data/certs/
```

On your local/AWS machine, go to the **"path-to-the"/local_data/certs** folder that was mounted to the docker_data folder previously in Section. 1, you should be able to see the server.crt and server.key files.

In addition, the CA certificate file (**cacert.pem**), located in the **/etc/ssl/misc/demoCA** directory, shall also be copied to **/docker_data/certs** (i.e., **"path-to-the"/local_data/certs**) for future use.

```
cp  /etc/ssl/misc/demoCA/cacert.pem  /docker_data/certs/
```

# 4 Configure Certificate on a Web Server (HTTPS)

## 4.1 Transport Certificate Files to Web Server Container

On your local/AWS machine, copy the server.crt and server.key to the **certs** folder under the **web_server** folder. Open another terminal and issue the following commands (make sure that the folders are correct entered based on your settings).

```
cd Docker_Security_Labs/
# Make sure to use the right path to enter this folder;
# It is in the folder where you issued the git clone command.

cp ciphers/local_data/certs/server.*  web_server/certs/
```

## 4.2 Launch Web Server

Next, start a web server on a container instance in the interactive mode.

```
cd web_server
docker build -t httpd-image .  # Note that there is a dot in the end
docker run -it --name web -p 8080:80 -p 443:443 httpd-image
```

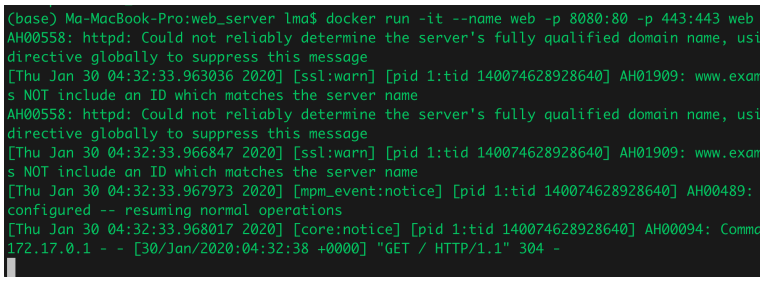



Figure 8: Web sever initialized.

If your container instance terminates immediately, you may want to try to launch it again as follows.

```
docker stop web
docker rm web
docker run -it --name web -p 8080:80 -p 443:443 httpd-image
```

### 4.3 Test HTTPs Connection

Open a browser (e.g., Firefox) on your local computer, type **http://127.0.0.1:8080** in the URL bar (for AWS VM, you should type **http://VM-public-IP:8080**; also you will need to adjust the security policies to allow traffic to port 8080 onto your VM). You should see a "Hello World" page.

Next, type **https://127.0.0.1** in the URL bar (for AWS VM, you should type **https://VM-public-IP**; also you will need to adjust the security policies to allow traffic to port 443 onto your VM).

#### 4.3.1 Did you receive a warning message for using https? Provide a screenshot. Why?

## 5 Install CA Certificate on Clients

In the previous section, you were warned about the security of the website. In this section, we will eliminate the warning message via applying the chain of trust.

### 5.1 Install CA Certificate on Firefox

Transport the CA certificate (i.e., the **cacert.pem** file) to your local computer (cacert.pem should be in the **/docker_data/certs/** folder). For AWS users, you will need to use sftp to download the file to your local computer (example shown in Fig. 9).

After downloading the CA certificate file, you can import it to a browser or even an operating system. For recent versions of Firefox, Go to Preferences -> Security & Privacy -> Certificates,

```
Ma-MacPro:~ lma$ sftp -i "/Users/lma/Documents/AWS/my-course-VM-key.pem" ubuntu@3.87.65.156
Connected to 3.87.65.156.
sftp> cd Docker_Security_Labs/ciphers/local_data/certs
sftp> get cacert.pem
```

Figure 9: Example sftp commands.

as shown in Fig. 10. Next, import your cacert.pem file. Choose "Trust this CA to identify web sites".

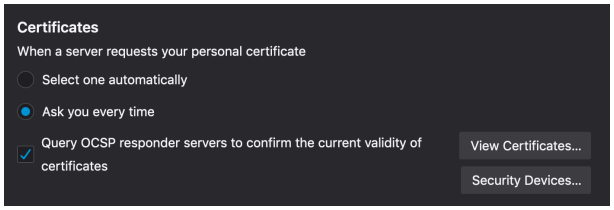



Figure 10: Security & Privacy in Newer Versions Firefox.

Now, test a https connection to your web server. If you were not warned about the site certificate, the root certificate has been successfully installed and functioning normally.

### 5.1.1 Explain why you did not receive the warning message?

The following openssl commands may give you additional insights to what have happen inside the browser verifying the web server's certificate.

```
openssl s_client -CAfile cacert.pem -connect 127.0.0.1:443
# For local users

openssl s_client -CAfile cacert.pem -connect VM-public-IP:443
# For AWS users
```

An example screenshot is give in Fig. 11.

```
(base) TCULT7BJHD3:certs lma$ openssl s_client -CAfile cacert.pem -connect 127.0.0.1:443
CONNECTED(00000003)
Can't use SSL_get_servername
depth=1 C = AU, ST = TX, O = TCU, OU = CS, CN = tcu.edu, emailAddress = df
verify return:1
depth=0 C = AU, ST = TX, L = FW, O = TCU, OU = COSC, CN = 127.0.0.1, emailAddress = fg
verify return:1
---
Certificate chain
 0 s:C = AU, ST = TX, L = FW, O = TCU, OU = COSC, CN = 127.0.0.1, emailAddress = fg
   i:C = AU, ST = TX, O = TCU, OU = CS, CN = tcu.edu, emailAddress = df
```

Figure 11: OpenSSL verification of server certificate.

## 5.2 Install CA Certificate on Windows (Optional)

First, you need to convert the certificate from PEM to DER format for Windows. An example command is give below.

```
openssl x509 -in cacert.pem -inform PEM -out my-rootCA.der -outform DER
```

This will create a DER format certificate that can be installed on Windows.

Transfer the DER format certificate to a Windows machine. Double click the certificate to install it. Follow the default option should be okay.

### 5.2.1 Test an SSL/TLS connection using IE or Edge. Shall you receive a warning message? Why?

If the CA certificate has been successfully installed, you shall NOT receive a warning message.

### 5.2.2 Test an SSL/TLS connection using Firefox. Explain what happened. Why?

## 5.3 AWS Web Server

### 5.3.1 What need to be done differently during the certificate creation process to make the whole thing work on a Docker web server instance running on an AWS virtual machine?

In addition, you will need to allow access to ports 80 (HTTP) and 443 (HTTPS) in the Security Groups configuration of your AWS virtual machine.

## 5.4 Chain of Trust Verification

You can verify the chain of trust using the openssl command line tool as well.

```
openssl verify -verbose -CAfile cacert.pem server.crt
openssl verify -purpose sslserver -CAfile cacert.pem server.crt
```

Here is a more complicated scenarios. Assume that you have created and signed an intermediate certificate. You used the intermediate certificate to signed on a user certificate. You also can verify the chain of trust among the three certificates using the following command.

```
openssl verify -verbose -CAfile <(cat Intermediate.pem RootCert.pem) UserCert.pem
# This command is to verify a third certificate
# For your reference
```

# 6 General Questions

### 6.0.1 Any corrections or general comments about this lab?

### 6.0.2 How long did it take you to complete this lab?

### 6.0.3 Was it an appropriate length lab?